Problem Set 03

Algorithm Analysis & Asymptotic Growth

Computer Science & Programming (Python)
SUPSI

Instructions

- Duration: 3 hours (in-class).
- You may use your notes and slides.
- Show all reasoning clearly.
- When asked to justify complexity, explain your reasoning.
- Unless stated otherwise, focus on worst-case upper bounds.

Exercise 1 — Summation and Exact Counting

Consider the following function:

```
def compute(n):
    s = 0
    for i in range(1, n+1):
        for j in range(i):
            s += 1
    return s
```

1. How many times does the inner statement execute?
2. Write the exact summation expression for the total number of iterations.
3. Simplify the summation into a closed-form expression.
4. Give the asymptotic upper bound of $T(n)$.
5. Is the algorithm linear, quadratic, or something else? Justify.

Exercise 2 — Early Termination and Case Analysis

Consider the function:

```
def has_duplicate(data):
    n = len(data)
    for i in range(n):
        for j in range(i+1, n):
            if data[i] == data[j]:
                return True
    return False
```

1. What is the worst-case time complexity?
2. What is the best-case time complexity?
3. Give an example input that achieves the best case.
4. Explain why the average case is still quadratic.
5. Suggest conceptually how this algorithm could be improved.

Exercise 3 — Mixed Linear and Logarithmic Growth

Consider:

```
def strange(n):
    total = 0
    i = 1
    while i < n:
        for j in range(n):
            total += 1
        i *= 2
    return total
```

1. How many times does the `while` loop execute?
2. How many times does the inner statement execute overall?
3. Give the asymptotic upper bound.
4. Compare this complexity to $O(n^2)$. Which grows faster?

Exercise 4 — Designing an $O(n \log n)$ Algorithm

You are given a list of integers `data`. Determine whether there exist two distinct elements whose sum is exactly zero.

Example:

```
data = [3, -1, 4, -3, 2]
# Output: True  (because 3 + (-3) = 0)
```

Part A — Naive Approach

1. Write a naive algorithm to solve the problem.
2. What is its worst-case time complexity?
3. Explain your reasoning.

Part B — Improved Algorithm

Design an algorithm whose worst-case time complexity is at most:

$$O(n \log n)$$

You may:

- Use sorting.
- Use binary search.
- Use built-in `sort()`.

1. Describe your algorithm clearly (in words or pseudocode).
2. Justify its asymptotic upper bound.
3. Explain why it is strictly better than the naive solution.

Exercise 5 — Growth Comparison and Dominance

Order the following functions from slowest growth to fastest growth:

$$\log n, \quad n, \quad n \log n, \quad n^2, \quad n^3, \quad 2^n$$

Then answer:

1. Is $10n^2$ asymptotically different from n^2? Why?
2. Is $n \log n$ asymptotically closer to n or to n^2? Justify.
3. For very large n, which grows faster: n^3 or 2^n?

Exercise 6 — Constrained Design Problem

You are given a list of student grades (integers between 1 and 6).

Design an algorithm that determines whether the list contains **at least three identical grades**.

Constraints:

- Worst-case complexity must be at most $O(n \log n)$.
- You may not use dictionaries.
- You may sort the list.

1. Describe your algorithm.
2. Provide pseudocode or Python code.
3. Prove its worst-case complexity.
4. Is it possible to do better than $O(n \log n)$? Why or why not?